Hospital Management System on Kubernetes: all three phases

One ASP.NET MVC application + SQL Server, deployed as two hospitals (A = Heilbronn Hospital, B = Hospital B). Same image everywhere: **mukhann/hospital-web:v2**. Each phase adds a layer on top of the one before. Counts and sizes are estimates.

new in this phase · carried over from the previous phase · learn-only or optional · traffic or data flow · control, scrape, admission check or backup · `file.yaml` the file in your repo that implements it

How your code becomes a running pod (the delivery path, all phases)

GitHub repository
src/ ASP.NET MVC app · **tests/** xUnit
Dockerfile multi-stage, non-root 1654, port 8080
docker-compose.yml (your Docker setup)
P1 **deploy/k8s/** plain YAML + docs
P2 **deploy/helm/hospital/** chart + values per hospital
P3 **deploy/argocd/** Application / ApplicationSet

push / pull request →

GitHub Actions (your DevSecOps pipeline)
existing: **ci** build + test · **CodeQL** SAST · **dependency scan** · **Gitleaks** secrets · **Sonar** · **Trivy** file system + image · **ZAP** DAST · **docker publish** build, scan, push
P2 **k8s_validate_devsecops.yml**: helm template → kubeconform (schema) → kube-linter (best practice) → trivy config (misconfiguration)
P3 **cosign sign** the pushed image; Kyverno later verifies the signature in the cluster

push image →

Docker Hub
mukhann/hospital-web :v1 :v2
built by the kubelet on whichever node runs the pod
SQL Server comes from **mcr.microsoft.com/mssql/server:2022-latest**
Private repo? create an *imagePullSecret* (see README)

PHASE 1 · Foundation

k3d cluster on your Linux machine · goal: deploy two hospitals · plain YAML, applied with **kubectl apply -f folder/**

3 nodes · 2 hospitals · 6 app pods + ~7 system ≈ 13 pods · ≈ 8 GB RAM

kubectl (your remote control)
apply, get, describe, logs, exec, port-forward, top, rollout, scale
talks to the API server on the control plane

API calls →

K3D CLUSTER "HOSPITAL" — K3S (SMALL, REAL KUBERNETES) RUNNING INSIDE DOCKER CONTAINERS

Node 1 · server-0 · control plane
API server front desk · **scheduler** picks a node for each pod · **controller manager** keeps "wanted = running" · **datastore** (SQLite) remembers state · also runs pods
`00-cluster/k3d-config.yaml`

Node 2 · agent-0 · worker
kubelet starts and checks pods · **kube-proxy** makes Service addresses work · **contained** runs the containers

Node 3 · agent-1 · worker
same components as agent-0
The scheduler spreads the pods over the 3 nodes; you do not choose the node (Phase 3 will)

Your browser / curl
http://hospital-a.localhost:8088
http://hospital-b.localhost:8088
*.localhost points to your own computer, no /etc/hosts edit needed

HTTP :8088 →

k3d load balancer
a Docker container (k3d-hospital-serverlb) host port **8088** → cluster port **80**

Namespace kube-system built into k3d · ≈ 7 pods · nothing to install

- **Traefik** Ingress controller: reads Ingress rules, routes by hostname
- **CoreDNS** the phone book: "db" → pod IP, "web.hospital-b.svc" → Service IP
- **Metrics Server** live CPU / RAM numbers for kubectl top and the HPA
- **local-path-provisioner** turns a PVC into a folder on a node (a PV)
- **svclb-traefik ×3** a DaemonSet: one pod per node, exposes Traefik

Namespace learn on demand · not part of the hospitals
Examples you apply one at a time and delete (`01-learn-only/`):
static pod (file dropped on a node) · **ReplicaSet** ×2 · **ReplicationController** ×2 · **DaemonSet** (3 pods) · Services: **NodePort** 30080, **LoadBalancer** 8089, **ExternalName** · **VPA** (read-only example, needs an extra install)

Docker Hub
kubelet pulls **mukhann/hospital-web:v2** for every web pod (both hospitals) and the SQL Server image for each db-0

pull image →

routes by host ↓ · CPU / RAM numbers ↑

Your files (what you apply)
`deploy/k8s/02-hospital-a/` ← **kubectl apply -f**
`deploy/k8s/03-hospital-b/` ← same files, other names
`02-secret.yaml` you create from `.example`; Git ignores it

Namespace hospital-a Heilbronn Hospital

Ingress web `07-ingress.yaml`
host **hospital-a.localhost** → Service web:80 · class traefik

▼ Traefik forwards by hostname

Service web `06-web-service.yaml`
ClusterIP :80 → pod port 8080 · Traefik sticky-session cookie (keeps a user on one pod)

▼ sends each request to a Ready pod

Deployment web → ReplicaSet → pods `05-web-deployment.yaml`
- **web pod 1** 1 container · mukhann/hospital-web:v2 :8080 · user 1654 · read-only fs · RAM folders: /tmp, login keys
- **web pod 2** same as pod 1
- **pod 3 – 4** only under load (HPA)

rolling update: maxUnavailable 0, maxSurge 1 (new pod ready first) · probes: **startup** + **readiness** /health (checks the DB), **liveness TCP** (DB outage must not restart pods) · requests 100m / 192Mi, limits 500m / 512Mi

▼ Server=db,1433 · CoreDNS turns "db" into the pod IP

Service db (headless) `03-db-service.yaml`
clusterIP None · :1433 · no Ingress, no NodePort: nothing outside the cluster can reach the database

▼

StatefulSet db → pod db-0 `04-db-statefulset.yaml`
- **db-0** SQL Server 2022 · user 10001 · requests 250m / 1.5Gi · limit 3Gi
- **probes** startup (port) · readiness (SELECT 1) · liveness (port)

▼ volumeClaimTemplates

PVC data-db-0 · 2 Gi → PV → a folder on the node
made by local-path-provisioner · mounted at /var/opt/mssql · survives pod deletion (deleting the namespace deletes it)

HPA web `08-hpa.yaml`
2 to 4 pods, target 70 % of the CPU request; reads numbers from the Metrics Server

ConfigMap web-config `01-configmap.yaml`
Hospital__Name · Database__MigrateOnStartup · Security__SecureCookies · Seed__AdminEmail · Seed__SampleData → **envFrom** into the web pods

Secret hospital-secret `02-secret.yaml`
MSSQL_SA_PASSWORD → web (inside the connection string) and db-0 · Seed__AdminPassword → web
base64 is not encryption

Namespace + labels `00-namespace.yaml`
app.kubernetes.io/name, instance, component, part-of: how Services and kubectl find their pods

Hardening
web: non-root, read-only file system, no Linux capabilities, no privilege escalation · db: non-root

Namespace hospital-b Hospital B

Ingress web `07-ingress.yaml`
host **hospital-b.localhost** → Service web:80 · class traefik

▼ Traefik forwards by hostname

Service web `06-web-service.yaml`
ClusterIP :80 → pod port 8080 · Traefik sticky-session cookie (keeps a user on one pod)

▼ sends each request to a Ready pod

Deployment web → ReplicaSet → pods `05-web-deployment.yaml`
- **web pod 1** 1 container · mukhann/hospital-web:v2 :8080 · user 1654 · read-only fs · RAM folders: /tmp, login keys
- **web pod 2** same as pod 1
- **pod 3 – 4** only under load (HPA)

rolling update: maxUnavailable 0, maxSurge 1 (new pod ready first) · probes: **startup** + **readiness** /health (checks the DB), **liveness TCP** (DB outage must not restart pods) · requests 100m / 192Mi, limits 500m / 512Mi

▼ Server=db,1433 · CoreDNS turns "db" into the pod IP

Service db (headless) `03-db-service.yaml`
clusterIP None · :1433 · no Ingress, no NodePort: nothing outside the cluster can reach the database

▼

StatefulSet db → pod db-0 `04-db-statefulset.yaml`
- **db-0** SQL Server 2022 · user 10001 · requests 250m / 1.5Gi · limit 3Gi
- **probes** startup (port) · readiness (SELECT 1) · liveness (port)

▼ volumeClaimTemplates

PVC data-db-0 · 2 Gi → PV → a folder on the node
made by local-path-provisioner · mounted at /var/opt/mssql · survives pod deletion (deleting the namespace deletes it)

HPA web `08-hpa.yaml`
2 to 4 pods, target 70 % of the CPU request; reads numbers from the Metrics Server

ConfigMap web-config `01-configmap.yaml`
Hospital__Name · Database__MigrateOnStartup · Security__SecureCookies · Seed__AdminEmail · Seed__SampleData → **envFrom** into the web pods

Secret hospital-secret `02-secret.yaml`
MSSQL_SA_PASSWORD → web (inside the connection string) and db-0 · Seed__AdminPassword → web
base64 is not encryption

Namespace + labels `00-namespace.yaml`
app.kubernetes.io/name, instance, component, part-of: how Services and kubectl find their pods

Hardening
web: non-root, read-only file system, no Linux capabilities, no privilege escalation · db: non-root

PHASE 2 · Security & packaging

same k3d cluster · goal: secure and standardise · hospitals now come from a **Helm chart**, plus monitoring and CI checks

3 nodes · 2 hospitals · ≈ 21 pods · ≈ 12 GB RAM

Carried over unchanged from Phase 1
k3d cluster with 3 nodes · kube-system (Traefik, CoreDNS, Metrics Server, local-path, svclb ×3) · browser → :8088 → Traefik · every Phase 1 object of each hospital (shown faded below). The Phase 1 folders stay in the repo as your plain-YAML reference; before **helm install** you delete the Phase 1 hospital (**kubectl delete namespace hospital-a**) because the names are the same.

Helm chart `deploy/helm/hospital/` NEW
The Phase 1 YAML turned into **templates** with blanks like `{{ .Values.hospitalName }}`
templates/ namespace · configmap · secret · db service + statefulset · web deployment + service · ingress · hpa · **networkpolicy** · **rbac** · **quota** · **limitrange** · **cronjob**
values.yaml defaults · **values-hospital-a.yaml** · **values-hospital-b.yaml** (name, hostname, image tag, e-mail)
`helm install hospital-a ./chart -f values-hospital-a.yaml` · `helm upgrade` / `helm rollback` (Helm remembers every release)

validates →

GitHub Actions `k8s_validate_devsecops.yml` NEW
on every pull request, before anything reaches a cluster:
1 - **helm template** renders the chart for A and B
2 - **kubeconform** is every object valid for this Kubernetes version?
3 - **kube-linter** best practice (limits, probes, non-root, …)
4 - **trivy config** misconfiguration scan of the YAML and the chart
→ fails the build on problems; works next to your existing CodeQL, Gitleaks, Trivy and ZAP workflows

Kustomize (learn-only)
Compared with Helm: patches plain YAML instead of templating it. Built into kubectl (`kubectl apply -k`).
Helm stays the main tool because it also gives releases, rollback and ready-made charts (the monitoring stack).

helm install hospital-a →

helm install hospital-b →

Namespace hospital-a Heilbronn Hospital · installed by **helm release hospital-a** (values-hospital-a.yaml)

FROM PHASE 1 (SAME OBJECTS, NOW RENDERED BY THE CHART)
Ingress hospital-a.localhost → Service web ClusterIP → web ×2 (HPA 2-4) non-root → Service db headless → db-0 SQL Server → PVC data-db-0 2 Gi
ConfigMap web-config · Secret hospital-secret · HPA · probes · requests / limits

NEW IN PHASE 2
- **NetworkPolicy ×4** NEW — **default deny** (all in + out) · allow Traefik → web:8080 · allow **web → db:1433** only · allow DNS (port 53) · allow backup job → db:1433 · Anything else, e.g. another namespace reaching the db, is dropped.
- **RBAC** NEW — ServiceAccount **web-sa** (no API token mounted) · Role **hospital-reader** (get/list pods, services, logs; **no secrets**) · RoleBinding to a read-only user: look, don't touch
- **Pod security** NEW — deeper securityContext (seccomp RuntimeDefault, drop ALL) + **Pod Security Standards** labels on the namespace (enforce baseline, warn restricted)
- **ResourceQuota + LimitRange** NEW — Quota caps this hospital's pods, CPU, RAM and PVCs, so it cannot starve the other one. LimitRange gives a default request / limit to any container that forgot to set one.
- **CronJob sql-backup (nightly) → PVC backups** NEW — a short-lived pod from an mssql-tools image runs `BACKUP DATABASE HospitalDB` against db-0:1433 (*dashed arrow*) and writes the .bak file to a second PVC **backups** (2 Gi); old files are pruned. Restore = `RESTORE DATABASE` from that file.

Namespace hospital-b Hospital B · installed by **helm release hospital-b** (values-hospital-b.yaml)

FROM PHASE 1 (SAME OBJECTS, NOW RENDERED BY THE CHART)
Ingress hospital-b.localhost → Service web ClusterIP → web ×2 (HPA 2-4) non-root → Service db headless → db-0 SQL Server → PVC data-db-0 2 Gi
ConfigMap web-config · Secret hospital-secret · HPA · probes · requests / limits

NEW IN PHASE 2
- **NetworkPolicy ×4** NEW — **default deny** (all in + out) · allow Traefik → web:8080 · allow **web → db:1433** only · allow DNS (port 53) · allow backup job → db:1433 · Anything else, e.g. another namespace reaching the db, is dropped.
- **RBAC** NEW — ServiceAccount **web-sa** (no API token mounted) · Role **hospital-reader** (get/list pods, services, logs; **no secrets**) · RoleBinding to a read-only user: look, don't touch
- **Pod security** NEW — deeper securityContext (seccomp RuntimeDefault, drop ALL) + **Pod Security Standards** labels on the namespace (enforce baseline, warn restricted)
- **ResourceQuota + LimitRange** NEW — Quota caps this hospital's pods, CPU, RAM and PVCs, so it cannot starve the other one. LimitRange gives a default request / limit to any container that forgot to set one.
- **CronJob sql-backup (nightly) → PVC backups** NEW — a short-lived pod from an mssql-tools image runs `BACKUP DATABASE HospitalDB` against db-0:1433 (*dashed arrow*) and writes the .bak file to a second PVC **backups** (2 Gi); old files are pruned. Restore = `RESTORE DATABASE` from that file.

scrapes container CPU / RAM / restarts ↑

Namespace monitoring one Helm release: kube-prometheus-stack NEW
- **Prometheus** scrapes container CPU / RAM / restarts (kubelet + kube-state-metrics) and node numbers (node-exporter); stores the time series
- **Grafana** dashboard "Hospital overview": pods up, restarts, CPU and RAM per hospital namespace; reads Prometheus
- **Alertmanager** one alert: "web pods not ready for 2 minutes" → e-mail / Slack / webhook
- **Prometheus Operator** manages the above through objects (ServiceMonitor, PrometheusRule)
- **kube-state-metrics** object state: replicas wanted vs ready, restarts
- **node-exporter ×3** a DaemonSet: CPU, RAM, disk of each node

≈ 8 pods · open Grafana with `kubectl port-forward` or an Ingress

PHASE 3 · Advanced / production-style

a **new kubeadm cluster on 2 VMs** (the k3d cluster is deleted first) · goal: multi-node, automation, resilience

2 VMs · 2 hospitals (+1 optional) · ≈ 40–45 pods · ≈ 12–16 GB RAM

GitOps: Git → Argo CD (namespace argocd) NEW
Git holds the chart, the values and the ApplicationSet. **Argo CD** pulls from Git, renders the Helm chart and applies it; if someone changes the cluster by hand it puts it back (self-heal).
ApplicationSet has a list (`hospital-a`, `hospital-b`) and generates **one Application per hospital**. A third hospital = one more line in the list (optional: +3 pods, +2 GB).
`deploy/argocd/applicationset.yaml` · you stop typing kubectl apply / helm install by hand

The front door: Gateway API + HTTPS NEW
Browser → **NodePort** on a VM (MetalLB optional) → **Traefik** installed with Helm (kubeadm has none built in) → **Gateway** (:443 TLS, :80 redirect) → one **HTTPRoute** per hospital → Service web.
cert-manager (3 pods): ClusterIssuer = local CA → Certificate → TLS Secret used by the Gateway. Then `Security__SecureCookies = "true"`.
Gateway API replaces the Ingress of Phases 1 and 2.

Supply chain NEW
GitHub Actions: build → Trivy image scan → push to Docker Hub → **cosign sign** (key). **Kyverno** in the cluster refuses any pod whose image is not signed by your key, or that runs as root, or has no limits.

Argo CD syncs chart + values ↓

HTTPS → Gateway → HTTPRoute ↓

Namespace hospital-a Heilbronn Hospital · created by an **Argo CD Application** (generated from the ApplicationSet)

THE HOSPITAL (PHASE 1 OBJECTS, FRONT DOOR CHANGED)
HTTPRoute https://hospital-a.<domain> + TLS → Service web ClusterIP → web ×2 (HPA 2-4) one per VM → Service db headless → db-0 SQL Server → PVC data-db-0 2 Gi

- **SealedSecret → Secret** NEW — the committed, encrypted **SealedSecret** is decrypted by the sealed-secrets controller into the normal Secret hospital-secret. No password file on anyone's disk.
- **Placement rules in the Deployment / StatefulSet** NEW — web: **topologySpreadConstraints** + pod anti-affinity (one pod per VM) · db-0: **nodeAffinity** to worker-1 (its volume is local to that VM)
- **PodDisruptionBudget web** NEW — **minAvailable: 1.** `kubectl drain` a VM and Kubernetes evicts web pods one at a time, never both. db-0 cannot move: its disk is local.
- **VolumeSnapshot data-db-0** NEW — point-in-time copy of the database disk through the snapshot-capable CSI class; restore into a new PVC. Velero also backs up this namespace into MinIO.
- **Admission by Kyverno** NEW — every pod here must be non-root, have limits, and use a **cosign-signed** image (mukhann/hospital-web:v2, signed by your pipeline).
- **Traces and cookies** NEW — web pods send OpenTelemetry traces to Jaeger (needs the app change) · `Security__SecureCookies="true"` because the site is now HTTPS

NetworkPolicies (enforced by Calico) · RBAC · ResourceQuota · LimitRange · backup CronJob · HPA 2-4 · Helm chart

Namespace hospital-b Hospital B · created by an **Argo CD Application** (generated from the ApplicationSet)

THE HOSPITAL (PHASE 1 OBJECTS, FRONT DOOR CHANGED)
HTTPRoute https://hospital-b.<domain> + TLS → Service web ClusterIP → web ×2 (HPA 2-4) one per VM → Service db headless → db-0 SQL Server → PVC data-db-0 2 Gi

- **SealedSecret → Secret** NEW — the committed, encrypted **SealedSecret** is decrypted by the sealed-secrets controller into the normal Secret hospital-secret. No password file on anyone's disk.
- **Placement rules in the Deployment / StatefulSet** NEW — web: **topologySpreadConstraints** + pod anti-affinity (one pod per VM) · db-0: **nodeAffinity** to worker-1 (its volume is local to that VM)
- **PodDisruptionBudget web** NEW — **minAvailable: 1.** `kubectl drain` a VM and Kubernetes evicts web pods one at a time, never both. db-0 cannot move: its disk is local.
- **VolumeSnapshot data-db-0** NEW — point-in-time copy of the database disk through the snapshot-capable CSI class; restore into a new PVC. Velero also backs up this namespace into MinIO.
- **Admission by Kyverno** NEW — every pod here must be non-root, have limits, and use a **cosign-signed** image (mukhann/hospital-web:v2, signed by your pipeline).
- **Traces and cookies** NEW — web pods send OpenTelemetry traces to Jaeger (needs the app change) · `Security__SecureCookies="true"` because the site is now HTTPS

NetworkPolicies (enforced by Calico) · RBAC · ResourceQuota · LimitRange · backup CronJob · HPA 2-4 · Helm chart

PHYSICAL VIEW: WHERE THE PODS REALLY RUN (2 VMS BUILT WITH MULTIPASS OR VAGRANT: KUBEADM INIT ON VM 1, KUBEADM JOIN ON VM 2)

kubeadm cluster Calico network plugin (enforces NetworkPolicy) · metrics-server · local-path StorageClass · all installed by you, k3s used to do this for you

VM 1 · cp-1 · control plane (taint removed, so it also runs workloads)
- **etcd** the cluster's notebook · *snapshot + restore drill*
- **kube-apiserver**
- **controller-manager**
- **kube-scheduler**

These four are **static pods**: the kubelet starts them from `/etc/kubernetes/manifests`, the same idea as `01-static-pod.yaml` in Phase 1.

- **kubelet** + containerd
- **kube-proxy**
- **calico-node**
- **coredns**
- **metrics-server**
- **web-a-1** hospital-a
- **web-b-1** hospital-b
- **platform tools** spread by the scheduler

VM 2 · worker-1
- **kubelet** + containerd
- **kube-proxy**
- **calico-node**
- **coredns**
- **node-exporter**
- **db-0 (hospital-a)** SQL Server · local volume on this VM
- **db-0 (hospital-b)** SQL Server · local volume on this VM
- **web-a-2** hospital-a
- **web-b-2** hospital-b
- **platform tools** spread by the scheduler

Both databases sit here because their volumes live on this VM's disk (node affinity). A drain of this VM cannot move them.

Taints & tolerations exercise: taint worker-1 → watch which pods stay away → add a toleration → remove the taint · **Node affinity** keeps db-0 on worker-1 · **Topology spread / anti-affinity** puts one web pod of each hospital on each VM · **PodDisruptionBudget** web minAvailable 1, tested with `kubectl drain` · **etcd backup** `etcdctl snapshot save` on cp-1

PLATFORM TOOLS (EACH INSTALLED FROM A READY-MADE HELM CHART, EACH IN ITS OWN NAMESPACE)

sealed-secrets NEW
Replaces the git-ignored secret file. You encrypt the password with `kubeseal` into a **SealedSecret** that is safe to commit. Only the controller in the cluster holds the private key and turns it into the real Secret.
Vault is learn-only: stronger but needs far more RAM and setup.

kyverno (policy engine) NEW
Admission checks on every new pod: must run as non-root · must have resource limits · image must carry your **cosign signature** · no :latest. Violations are rejected before they run.
Chosen over OPA/Gatekeeper because policies are plain YAML, not a new language (OPA is learn-only).

snapshots: CSI hostpath driver + snapshot controller NEW
k3d's local-path cannot snapshot. A light CSI driver adds a **VolumeSnapshotClass**. **VolumeSnapshot** of `data-db-0` = a point-in-time copy of the database disk; restore it into a new PVC.
Longhorn would also work but is too heavy for 2 VMs.

velero + MinIO NEW
MinIO = a local S3-style bucket. **Velero** backs up a whole namespace (objects + volume data) into it and can restore it into a fresh cluster. This is the disaster-recovery layer; the SQL CronJob from Phase 2 stays as the database-level backup.

tracing: OpenTelemetry + Jaeger NEW
The app gets OpenTelemetry packages (**a small code change**) and sends traces to **Jaeger**: you see one request go web → database with timings. Without the code change there is nothing to trace.

monitoring (From Phase 2, unchanged)
Prometheus, Grafana, Alertmanager, operator, kube-state-metrics, node-exporter (now ×2, one per VM). Also carried over: NetworkPolicies (now really enforced by Calico), RBAC, quotas, backup CronJob, Helm chart, CI checks.

What changes from phase to phase

	Phase 1 · Foundation	Phase 2 · Security & packaging	Phase 3 · Advanced
Cluster	k3d (k3s in Docker)	same k3d cluster	kubeadm on 2 VMs (a "real" cluster)
Nodes	3: 1 server + 2 agents	3	2: control plane + worker, both run pods
How hospitals are deployed	**kubectl apply -f** folder (plain YAML)	**helm install** chart + values file	**Argo CD ApplicationSet** from Git (GitOps)
Front door	Traefik Ingress, HTTP on :8088	same	Gateway API + HTTPS (cert-manager)
Secrets	Secret from a git-ignored file	same + RBAC limits who can read it	Sealed Secrets (encrypted, safe in Git)
Storage	local-path PVC per database	+ backup PVC filled by a nightly CronJob	CSI driver with VolumeSnapshot; Velero + MinIO
Network rules	everything can talk to everything	NetworkPolicy: default deny, only web → db	same, enforced by Calico
Pod security	non-root, read-only file system (web)	+ Pod Security Standards labels, ResourceQuota, LimitRange	+ Kyverno policies, signed images
Scale & availability	HPA 2 to 4 web pods; rolling update, rollback	same	+ spread across VMs, node affinity for db, PodDisruptionBudget, drain test
Observability	kubectl logs, top, describe	Prometheus, Grafana, Alertmanager	+ OpenTelemetry traces in Jaeger
Quality gates (CI)	your existing DevSecOps workflows	+ kubeconform, kube-linter, trivy config	+ cosign sign, Kyverno verify
Your files	**deploy/k8s/** (delivered)	**deploy/helm/**, monitoring values, CI workflow	**deploy/argocd/**, kubeadm notes, platform values
Pods (estimate)	**≈ 13 (6 app + 7 system)**	**≈ 21 (+ ≈ 8 monitoring)**	**≈ 40–45 (13 system, 6 app, ≈ 7 monitoring, 14–19 tools)**
RAM (estimate)	**≈ 8 GB**	**≈ 12 GB**	**≈ 12–16 GB for the two VMs, plus ≈ 3 GB for your computer**
Application code changes	none	none	OpenTelemetry packages (tracing); optionally shared login-cookie keys

Learn-only items (dashed) are explained with small example files but are not part of the running system. All pod counts and RAM figures are estimates and depend on replicas and component settings. Install the Phase 2 and 3 tools only when you reach their topic, and remove them again to free RAM.